Repaso del día 3

16 preguntas sobre cadena de bloques, consenso, generales bizantinos, Proof of Work y Proof of Stake

Las 16 preguntas — Repaso día 3

1. ¿Qué es el mempool y para qué sirve en una blockchain?
2. ¿Qué contiene la cabecera (header) de un bloque?
3. ¿Qué es la Merkle root y por qué se incluye en cada bloque?
4. ¿Cómo se enlazan los bloques entre sí para formar una cadena?
5. ¿Qué pasa si un atacante intenta modificar un dato de un bloque pasado?
6. ¿Qué tres problemas no resuelve la cadena de bloques por sí sola?
7. ¿Qué es un ataque Sybil y por qué hace insuficiente votar entre nodos?
8. ¿En qué consiste el problema de los generales bizantinos?

9. ¿Cuántos nodos honestos se necesitan para alcanzar consenso bizantino?

¿Por qué BFT clásico no encaja en una red abierta como Bitcoin?

¿Cuál es la idea central de Proof of Work y por qué resuelve el problema?

¿Qué es el nonce en Proof of Work y qué papel cumple?

¿Por qué se dice que Proof of Work es asimétrico?

Si dos mineros encuentran un bloque válido a la vez, ¿cómo se resuelve el fork?

¿Qué es el ataque del 51% y por qué no es viable hoy en Bitcoin?

¿En qué se diferencia Proof of Stake de Proof of Work?

PREGUNTA 1 / 16

El mempool

Pregunta

¿Qué es el mempool y para qué sirve en una blockchain?

Respuesta razonada

Es la 'sala de espera' de las transacciones. Cuando un nodo recibe una transacción válida (firma correcta, formato OK) la guarda en su mempool — un conjunto de transacciones validadas pero todavía no incluidas en ningún bloque. Cuando un minero quiere proponer un bloque, selecciona transacciones de su mempool. Una vez incluidas en un bloque confirmado, se eliminan del mempool. Es como una bandeja de entrada compartida (con matices) entre todos los nodos.

PREGUNTA 2 / 16

Anatomía de un bloque

Pregunta

¿Qué contiene la cabecera (header) de un bloque?

Respuesta razonada

Tres elementos clave: el hash del bloque anterior (el eslabón hacia atrás que crea la cadena), una marca de tiempo (timestamp) y la Merkle root (resumen criptográfico de todas las transacciones del bloque). El identificador del bloque es el hash de su cabecera. El cuerpo (body) contiene la lista completa de transacciones; la cabecera resume todo en muy pocos bytes.

PREGUNTA 3 / 16

Merkle root

Pregunta

¿Qué es la Merkle root y por qué se incluye en cada bloque?

Respuesta razonada

Es el hash raíz de un árbol binario donde cada hoja es el hash de una transacción y cada nodo interno es el hash de sus dos hijos. Resume todas las transacciones del bloque en un único hash de 32 bytes. Si cambia una sola transacción, la Merkle root cambia. Permite verificar la integridad del bloque sin recorrer todas sus transacciones, y demostrar la pertenencia de una transacción al bloque enviando solo log(N) hashes.

PREGUNTA 4 / 16

Encadenamiento de bloques

Pregunta

¿Cómo se enlazan los bloques entre sí para formar una cadena?

Respuesta razonada

Cada bloque guarda dentro de su cabecera el hash del bloque anterior (campo prev_hash). El bloque N tiene prev_hash = hash(N-1), el bloque N+1 tiene prev_hash = hash(N), y así hasta el bloque 0 (génesis), que no tiene prev_hash porque no hay bloque anterior. Es ese campo el que crea la cadena: cada bloque apunta criptográficamente al anterior.

PREGUNTA 5 / 16

Cascada de invalidación

Pregunta

¿Qué pasa si un atacante intenta modificar un dato de un bloque pasado?

Respuesta razonada

Al cambiar el dato, cambia el hash del bloque. Pero el bloque siguiente guardaba el prev_hash apuntando al hash original — ahora apunta a un bloque que ya no existe. La cadena se rompe en ese punto. Para que vuelva a ser coherente, el atacante tendría que rehacer todos los bloques posteriores hasta el actual. Modificar el pasado obliga a reescribir toda la historia posterior.

PREGUNTA 6 / 16

Las 3 preguntas incómodas

Pregunta

¿Qué tres problemas no resuelve la cadena de bloques por sí sola?

Respuesta razonada

Uno: ¿qué pasa si dos nodos crean bloques distintos al mismo tiempo? La red podría dividirse en dos cadenas. Dos: ¿qué impide que un atacante manipule su copia y regenere todos los hashes? Calcular hashes es rápido y barato. Tres: ¿cómo decidimos cuál es la cadena verdadera si hay varias circulando? Sin autoridad central, votar no funciona. Estos tres problemas exigen un mecanismo de consenso.

PREGUNTA 7 / 16

Ataque Sybil

Pregunta

¿Qué es un ataque Sybil y por qué hace insuficiente votar entre nodos?

Respuesta razonada

Es cuando un atacante crea muchas identidades falsas en la red para influir en cualquier votación. En un sistema sin autoridad central, cualquiera puede levantar miles de nodos en su ordenador con identidades distintas. Si el consenso fuera 'la mayoría de nodos decide', el atacante con más recursos siempre ganaría. Por eso los sistemas como Bitcoin no votan por nodo: votan por trabajo computacional o por capital comprometido.

PREGUNTA 8 / 16

Generales bizantinos

Pregunta

¿En qué consiste el problema de los generales bizantinos?

Respuesta razonada

Varios generales rodean una ciudad y deben decidir entre atacar o retirarse. Solo se comunican mediante mensajeros, y tanto generales como mensajeros pueden ser traidores. Hay un comandante que da la orden y tenientes que actúan. El éxito es que todos los leales tomen la misma decisión; el fracaso es que unos ataquen y otros se retiren. La pregunta es cuántos traidores se pueden tolerar para que el consenso siga siendo posible entre los leales.

PREGUNTA 9 / 16

Regla BFT

Pregunta

¿Cuántos nodos honestos se necesitan para alcanzar consenso bizantino?

Respuesta razonada

Lamport, Shostak y Pease demostraron en 1982 que para N nodos con F traidores hace falta N > 3F. Es decir, más de dos tercios deben ser honestos. Con 3 generales y 1 traidor no hay solución (3 no es mayor que 3). Con 4 y 1 sí (4 > 3). Con 6 y 2 no (6 no es mayor que 6). Con 7 y 2 sí (7 > 6). Esta es la base de los algoritmos BFT (Byzantine Fault Tolerant) que usan Hyperledger Fabric, Tendermint y similares.

PREGUNTA 10 / 16

BFT en redes abiertas

Pregunta

¿Por qué BFT clásico no encaja en una red abierta como Bitcoin?

Respuesta razonada

BFT exige conocer de antemano la lista de participantes con identidades fijas. En una red abierta cualquiera puede entrar, así que un atacante puede crear miles de identidades falsas (Sybil) y conseguir más de un tercio sin esfuerzo. Además, el coste de mensajes en BFT crece como N^2 o peor, lo que limita la escalabilidad. BFT funciona muy bien en redes permisionadas (todos identificados, como Fabric) pero no en redes públicas sin permisos.

PREGUNTA 11 / 16

La idea de Proof of Work

Pregunta

¿Cuál es la idea central de Proof of Work y por qué resuelve el problema?

Respuesta razonada

Si no podemos confiar en las identidades, atémonos a algo que no se puede falsificar: el coste computacional. Proponer un bloque debe ser caro (mucho cómputo, mucha electricidad), ajustable (un bloque cada cierto tiempo) y verificable instantáneamente. Quien hace el trabajo gana una recompensa, así que ser honesto compensa más que atacar. Atacar exige rehacer todo el trabajo computacional posterior — económicamente inviable a escala.

PREGUNTA 12 / 16

El nonce

Pregunta

¿Qué es el nonce en Proof of Work y qué papel cumple?

Respuesta razonada

Es un número arbitrario que se añade a la cabecera del bloque. El minero lo va variando una y otra vez y recalcula el hash de la cabecera buscando que cumpla cierta condición (por ejemplo, que empiece con suficientes ceros, o que sea menor que un target). Encontrar un nonce válido cuesta millones o billones de intentos. Es la 'lotería' que define quién propone el siguiente bloque. Verificar que el nonce es correcto, en cambio, cuesta una sola operación.

PREGUNTA 13 / 16

La asimetría de PoW

Pregunta

¿Por qué se dice que Proof of Work es asimétrico?

Respuesta razonada

Porque encontrar un bloque válido es muy costoso, pero verificar que lo es resulta trivial. El minero hace millones de cálculos hasta dar con un nonce que cumpla la condición; cualquier otro nodo, al recibir el bloque, hace UN solo cálculo de hash y comprueba si es válido. Esa asimetría — caro de producir, barato de verificar — es lo que permite a la red detectar bloques inválidos sin gastar apenas recursos.

PREGUNTA 14 / 16

Cadena más larga

Pregunta

Si dos mineros encuentran un bloque válido a la vez, ¿cómo se resuelve el fork?

Respuesta razonada

La regla de oro es: gana la cadena con más trabajo acumulado. Cada uno de los dos mineros difunde su bloque y la red se bifurca temporalmente. Cuando alguno de los dos lados encuentra el siguiente bloque (N+2), su cadena se vuelve más larga. Todos los nodos abandonan la cadena corta y adoptan la larga; las transacciones de la rama descartada vuelven al mempool. Es probabilístico: con cada bloque adicional, revertir un bloque anterior se vuelve exponencialmente más difícil.

PREGUNTA 15 / 16

Ataque del 51%

Pregunta

¿Qué es el ataque del 51% y por qué no es viable hoy en Bitcoin?

Respuesta razonada

Si alguien controla más del 50% del cómputo de la red, puede generar bloques más rápido que el resto, construir una cadena alternativa más larga e imponerla como la verdadera, reescribiendo transacciones recientes (doble gasto). En Bitcoin hoy exigiría inversión en hardware y electricidad por miles de millones de dólares. Y si lo hiciera, derrumbaría la confianza en Bitcoin: su 'inversión' valdría cero. La economía protege la red más que la criptografía: ningún actor racional tiene incentivos para atacar.

PREGUNTA 16 / 16

Proof of Stake

Pregunta

¿En qué se diferencia Proof of Stake de Proof of Work?

Respuesta razonada

En PoS los nodos no compiten por gasto computacional sino bloqueando monedas (stake). Los validadores se eligen de forma pseudoaleatoria proporcional a su stake. Si actúan honestamente reciben recompensas; si atacan o fallan, pierden parte del stake (slashing). Ventajas: produce bloques más rápido y consume mucha menos electricidad. Inconveniente: los hackers necesitan el 51% de las monedas en lugar del 51% del cómputo. Ethereum migró de PoW a PoS en 2022.

Bajar el whitepaper de Satoshi

https://github.com/jjnieto/Curso-HLFabric/tree/main/Modulo-1

Bitcoin: Un Sistema de Efectivo Electrónico Usuario-a-Usuario

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Traducido al Español de bitcoin.org/bitcoin.pdf
por Angel León - www.diariobitcoin.com

Abstracto. Una versión puramente electrónica de efectivo permitiría que los pagos en línea fuesen enviados directamente de un ente a otro sin tener que pasar por medio de una institución financiera. Firmas digitales proveen parte de la solución, pero los beneficios principales se pierden si existe un tercero confiable para prevenir el doble-gasto. Proponemos una solución al problema del doble gasto utilizando una red usuario-a-usuario. La red coloca estampas de tiempo a las transacciones al crear un hash de las mismas en una cadena continua de pruebas de trabajo basadas en hashes, formando un registro que no puede ser cambiado sin volver a recrear la prueba de trabajo. La cadena más larga no solo sirve como la prueba de la secuencia de los eventos testificados, sino como prueba de que vino del gremio de poder de procesamiento de CPU más grande. Siempre que la mayoría del poder de procesamiento de CPU esté bajo el control de los nodos que no cooperan para atacar la red, estos generarán la cadena más larga y le llevarán la ventaja a los atacantes. La red en sí misma requiere una estructura mínima. Los mensajes son enviados bajo la base de mejor esfuerzo, y los nodos pueden irse y volver a unirse a la red como les parezca, aceptando la cadena de prueba de trabajo de lo que sucedió durante su

Trilema de Blockchain

El trilema de Blockchain

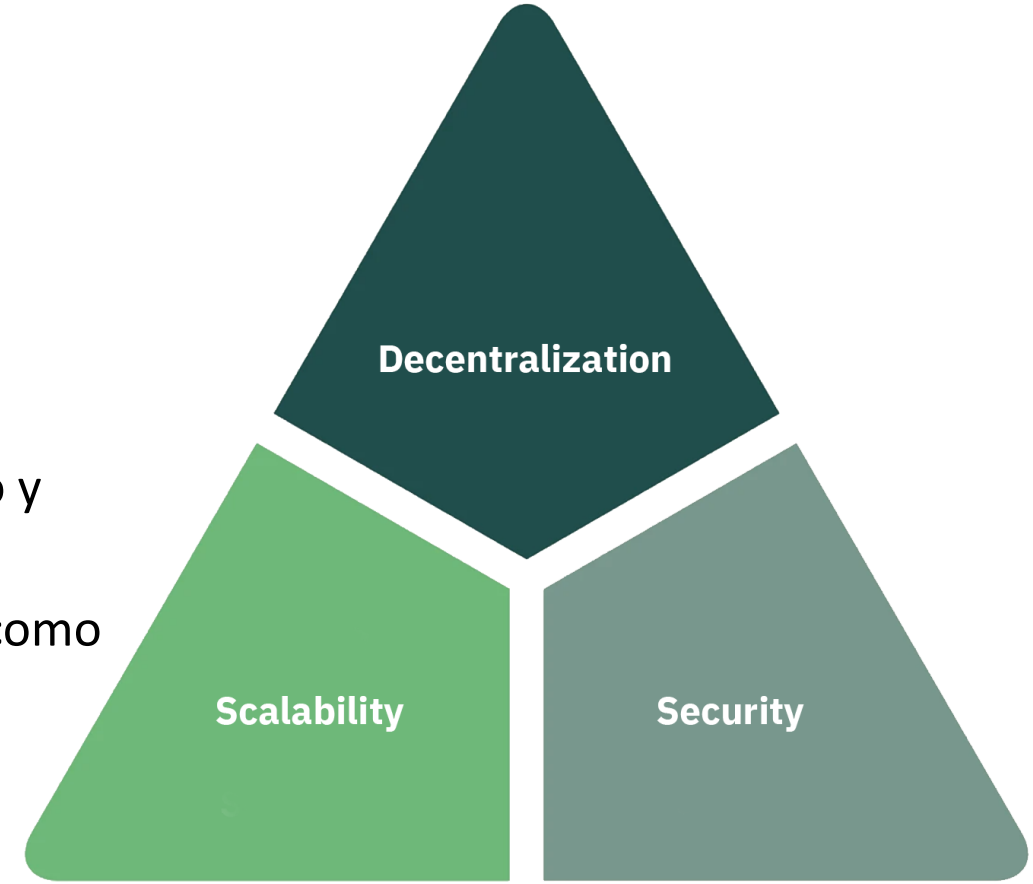
¿Qué es el trilema de Blockchain?

Propuesto por Vitalik Buterin (Ethereum).

Una blockchain busca tres propiedades:

1. Seguridad → resistente a ataques y fallos.
2. Escalabilidad → procesar muchas transacciones rápido y barato.
3. Descentralización → que cualquiera pueda participar como nodo sin barreras.

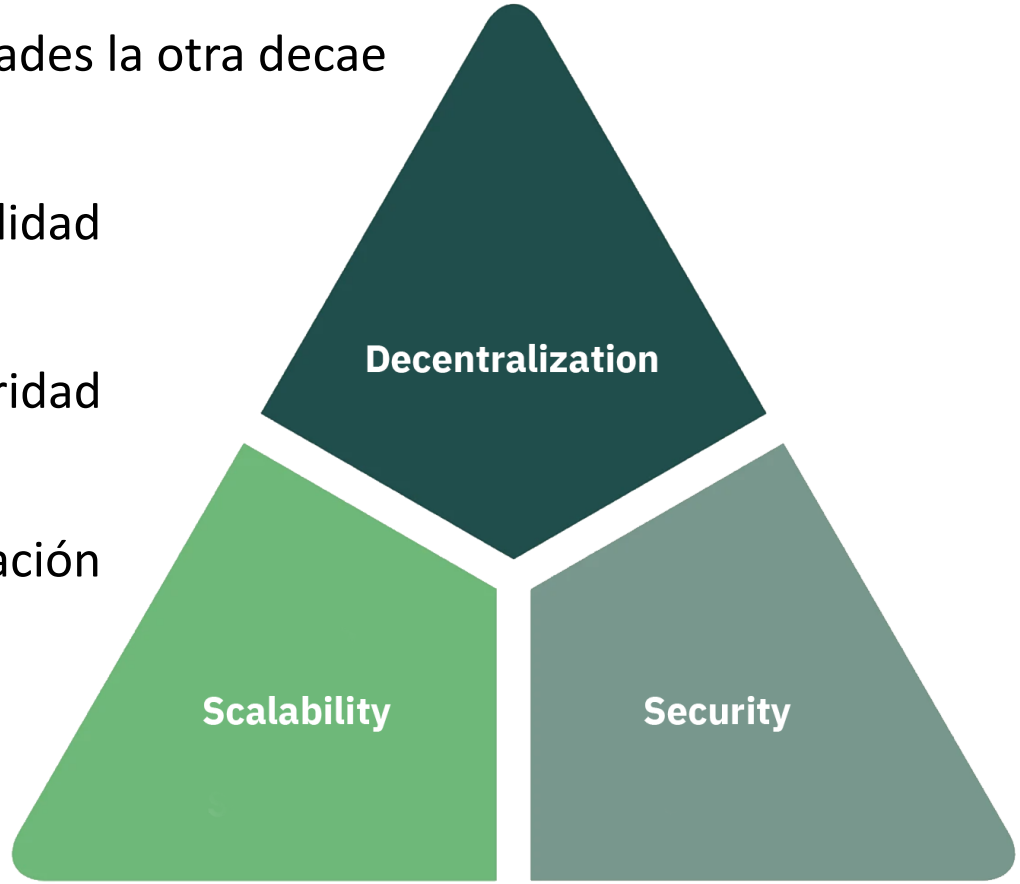
Problema: en la práctica es muy difícil optimizar las tres al mismo tiempo.



Actividad – Trilema de Blockchain

Debatir entre los alumnos por qué si optimizas dos propiedades la otra decae

1. Alta descentralización y alta seguridad => Baja escalabilidad
2. Alta descentralización y alta escalabilidad => Baja seguridad
3. Alta seguridad y alta escalabilidad => Baja descentralización



Actividad – Trilema de Blockchain

Alta descentralización y alta seguridad => Baja escalabilidad

1. Validación redundante y pesada
    1. En una red muy descentralizada cada nodo completo valida todas las transacciones.
    2. Eso asegura máxima seguridad (todos comprueban todo), pero implica que la velocidad está limitada por el nodo más lento.
    3. Resultado: pocas transacciones por segundo.
2. Bloques pequeños y tiempos largos
    1. Para que cualquier nodo (incluso con conexión débil) participe, los bloques deben ser pequeños y espaciados en el tiempo.Seguridad y descentralización se mantienen, pero el sistema procesa muy poco volumen.
    2. Ejemplo: Bitcoin (~7 TPS, bloques cada 10 minutos).
3. 3. Coste de sincronización
    1. Miles de nodos repartidos por el mundo necesitan recibir y verificar cada bloque.
    2. Si aumentas la frecuencia o el tamaño, muchos nodos no podrían seguir el ritmo → perderías descentralización.
    3. Para mantenerla, reduces velocidad.

Ejemplo Ethereum y Bitcoin

Ambas han precisado de sidechains o Layer 2 para agilizar y abaratar las transacciones sufriendo periodos de congestión

Actividad – Trilema de Blockchain

Alta descentralización y alta escalabilidad => Baja seguridad

1. Muchos nodos pequeños y rápidos → difícil coordinar seguridad
    1. Si cualquiera puede montar un nodo con pocos recursos (alta descentralización), y además la red procesa miles de transacciones por segundo (alta escalabilidad), cada nodo debe validar muy rápido y con poco coste.
    2. Resultado: validaciones menos rigurosas o más superficiales → se abre la puerta a ataques (transacciones inválidas colándose, forks mal gestionados).
2. 2. Hardware ligero → nodos vulnerables
    1. Para escalar con miles de TPS, la red puede relajar requisitos de validación.
    2. Eso sacrifica seguridad: basta comprometer pocos nodos “fuertes” para manipular la red

Ejemplo Solana (2021)

Buscaba escalabilidad alta (miles de TPS) y aumentar el número de nodos. Finalmente sufrió caídas de seguridad por saturación durante 17 horas al no ser capaz de eliminar un fork.

Actividad – Trilema de Blockchain

Alta seguridad y alta escalabilidad => Baja descentralización

1. Hardware exigente
    1. Para procesar miles de transacciones por segundo de forma segura, cada nodo necesita gran capacidad de cómputo y ancho de banda.
    2. Resultado: solo unos pocos actores con infraestructura potente pueden ser validadores.
2. Coste de entrada alto
    1. Cuanto más compleja y rápida es la validación, mayor es el coste de participar.
    2. Esto excluye a nodos pequeños y reduce el número de participantes.
    3. Consecuencia: red más centralizada en pocas manos.
3. Riesgo de concentración
    1. Aunque la red sea segura y rápida, si solo un grupo reducido de validadores controla el proceso, se compromete la descentralización.
    2. Posibles efectos: censura de transacciones, colusión entre validadores, puntos únicos de fallo.

Ejemplo: Redes permissioned (empresariales)

Muy rápidas y seguras, pero solo un conjunto limitado de nodos autorizados participa.Seguridad y escalabilidad altas, pero descentralización sacrificada por diseño.

Conclusiones – Trilema de Blockchain

Alta descentralización y alta seguridad => Baja escalabilidad

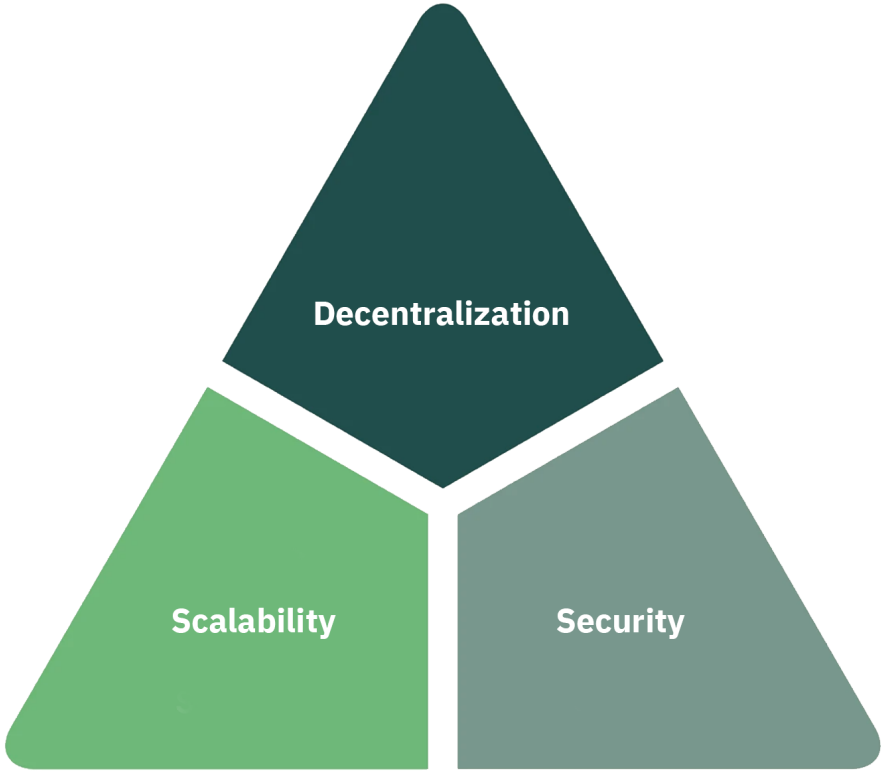
“Cuando quieres que muchos nodos distintos verifiquen cada detalle con la máxima seguridad, la red se vuelve lenta y cara: esa es la factura de combinar descentralización y seguridad.”

Alta descentralización y alta escalabilidad => Baja seguridad

“Si abres demasiado la red (cualquiera entra) y además la haces muy rápida (miles de transacciones), la validación se vuelve más débil y la seguridad baja. Se pierde control sobre quién valida y sobre la calidad de la validación.”

Alta seguridad y alta escalabilidad => Baja descentralización

“Si quieres seguridad fuerte y procesar miles de transacciones por segundo, la validación exige nodos potentes y caros. Eso concentra el control en pocos actores y reduce la descentralización.”



♬Soy Minero♫

El poder de los mineros

MEV, front-running y las soluciones que está aplicando la industria

RECORDATORIO

El problema: MEV

¿Qué es MEV?

MEV (Maximal Extractable Value) es el beneficio que un minero o validador puede extraer reordenando, incluyendo o excluyendo transacciones del bloque que produce. No es un fallo: es una consecuencia de que UN solo actor decida qué entra y en qué orden.

Tipos de ataques

Front-running: el minero ve una transacción grande en el mempool y mete la suya antes para aprovechar el cambio de precio.
Back-running: ejecuta una transacción justo después de otra que mueve el mercado.
Sandwich attack: compra antes y vende justo después de una transacción grande, atrapando al usuario en medio.

OVERVIEW

Dos grandes familias de soluciones

La industria está atacando el problema desde dos ángulos complementarios.

1. Separar quién ordena de quién propone

Proposer-Builder Separation (PBS)

El validador deja de elegir el contenido del bloque. Construye el bloque un actor especializado (builder) y el validador solo lo firma. Idea: si el validador no ve las transacciones, no puede aprovecharse de ellas.

2. Esconder qué se está enviando

Encrypted Mempools

Las transacciones viajan cifradas por el mempool. Nadie (ni siquiera el validador) puede leerlas hasta que se incluyen en el bloque. Idea: si el contenido es opaco, no se puede hacer front-running ni sandwich.

SOLUCIÓN 1

Proposer-Builder Separation (PBS)

La idea

Hasta ahora un único actor (el validador o minero) decidía qué transacciones incluir y en qué orden. Tenía el monopolio del MEV. PBS rompe ese monopolio dividiendo el rol en dos:

Builder (constructor): actor especializado en armar bloques rentables. Recoge transacciones del mempool, las ordena (incluso compitiendo por MEV) y arma un bloque candidato. Es un mercado abierto: cualquiera puede ser builder.

Proposer (proponedor): es el validador. Solo recibe bloques ya armados de varios builders, elige el que más le paga y lo firma. Nunca ve el contenido en detalle hasta después.

SOLUCIÓN 1

PBS — quién es quién y qué gana cada uno

Aclaración: *desde The Merge (2022) Ethereum es Proof of Stake. Ya no hay minería ni mineros. Los validadores (32 ETH en stake) son seleccionados aleatoriamente cada slot (12 s) para hacer de proposer. La 'recompensa del bloque' (ETH recién emitido) ya no la cobra un minero, la cobra el proposer.*

PROPOSER

Validador con 32 ETH en stake, elegido para este slot

¿Qué gana?

- Recompensa del bloque (ETH nuevo emitido)
- Tips/fees de transacciones
- Pago del builder ganador (la mayor parte del MEV)

En PBS no construye el bloque: solo firma a ciegas la cabecera del que más le paga.

BUILDER

Especialista en construir bloques rentables (MEV incluido)

¿Qué gana?

- El MEV que extrae del bloque MENOS
- Lo que paga al proposer

La competencia entre builders aprieta el margen: para ganar la subasta tienen que pagar casi todo el MEV al proposer.

RELAY

Intermediario neutral entre builders y proposer

¿Qué gana?

- Reputación (Flashbots, BloXroute…)
- A veces pequeñas tarifas

Guarda los bloques de los builders, muestra al proposer solo cabecera y precio. Revela el contenido solo tras la firma. Su valor es la confianza.

SEARCHER

Detecta oportunidades concretas de MEV

¿Qué gana?

- Margen del MEV que descubre MENOS
- Lo que paga al builder

Envía 'bundles' (transacciones específicas) a los builders. Si el builder los incluye y ganan, todos cobran. Si no, nada.

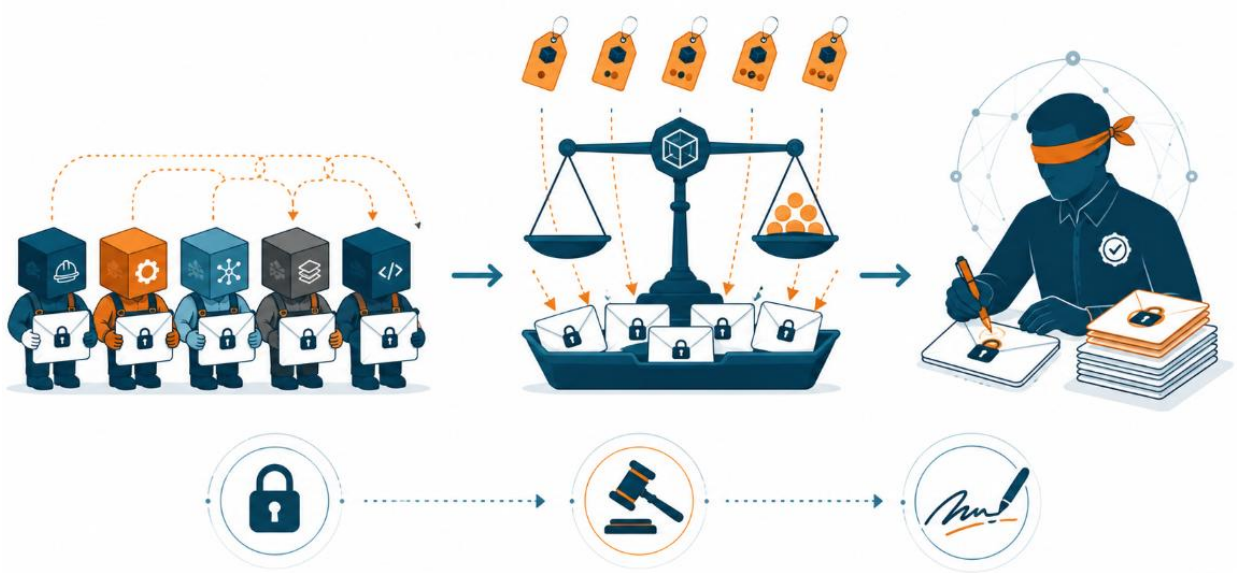
SOLUCIÓN 1

PBS — paso a paso

El flujo

1. Los usuarios envían sus transacciones al mempool (o a un mempool privado).
2. Múltiples builders compiten por construir el bloque más rentable (incluyendo MEV legítimo).
3. Cada builder envía a un relay un bloque sellado y el precio que paga al proposer por incluirlo.
4. El relay muestra al proposer solo la cabecera y el pago — no el contenido.
5. El proposer firma la cabecera del bloque que más le paga. A partir de ahí, el relay revela el contenido y se publica en la red.

Resultado: el validador no puede hacer front-running porque firma a ciegas. Si lo intenta, no le firmarían los relays.



SOLUCIÓN 2

EIP-8184 LUCID — encrypted mempool

La idea

PBS resuelve el problema desde el lado del validador. Los encrypted mempools lo atacan desde el lado del usuario: si el contenido de la transacción está cifrado mientras viaja por el mempool, nadie puede hacer front-running porque nadie sabe qué hay dentro.

El reto técnico

El truco está en cifrar la transacción de modo que pueda incluirse en un bloque ANTES de descifrarla. Solo después de que el bloque esté ordenado y firmado, se descifra y se ejecuta. Si el validador no quiere incluirla porque no la entiende, mala suerte: ya está su firma comprometida con el bloque.

ACTIVIDAD

¿Cómo intentarías hacer trampa con cada solución?

En grupos de 3-4 personas, durante 10 minutos. Pensad como un atacante que quiere hacer front-running. ¿Qué jugada intentaríais con cada solución? ¿Funcionaría?

SOLUCIÓN 1 — PBS

¿Cómo intentarían hacer trampa?

Como builder malicioso: incluyo en mi bloque una transacción de front-running contra una operación grande del mempool, y compenso pagando al proposer un fee desproporcionado para que mi bloque sea el más rentable.

¿Funciona?

SÍ. PBS no impide que un builder haga MEV — solo separa quién construye de quién firma. La competencia entre builders fuerza a transferir casi todo el MEV al proposer, pero el **usuario sigue siendo víctima.**

SOLUCIÓN 2 — ENCRYPTED MEMPOOL

¿Cómo intentarían hacer trampa?

El front-running clásico no funciona: el contenido está cifrado, nadie ve qué transacción intenta hacer cada usuario. Sin información, no se puede hacer sandwich ni adelantar.

¿Vías indirectas?

Difíciles pero no imposibles:

- Comprometer al key publisher (descifrar antes de tiempo).
- Censurar transacciones (no incluirlas) — pero LUCID lo mitiga con inclusion lists.
- Análisis de patrones (tamaño, momento, gas) — el contenido no se ve pero los metadatos sí.

Conclusiones

1. El MEV no se 'arregla' eliminándolo — siempre que UN actor decida el orden, habrá MEV. La industria asume su existencia y trabaja para **redistribuirlo o democratizarlo.**

2. PBS ataca el problema desde el lado del validador (no decide qué incluye). **Encrypted mempools** lo atacan desde el lado del usuario (su contenido es opaco).

3. Las soluciones se **complementan**: en el futuro convivirán PBS, encrypted mempools y mecanismos de redistribución. No hay una bala de plata.

4. En redes **permissioned** (como Hyperledger Fabric) el problema cambia de naturaleza: los ordenadores son conocidos y responden a un consorcio, no son anónimos compitiendo por MEV. Aun así, la gobernanza de quién ordena sigue siendo crítica.

lake of shadows
S
TREASURE HUNT

ROT13

En el año 44 a.C., Julio César usaba un método de cifrado para comunicarse con sus generales: desplazaba cada letra del alfabeto un número fijo de posiciones. Este método se conoce como el **Cifrado César**.

El siguiente mensaje está cifrado con un desplazamiento de 13 posiciones (ROT13):

Ry cevzre oybdhr qr Ovgpbva fr yynzn Trarfvf Oybpx l shé zvanqb cbe Fngbfuv Anxnzbgb ry 3 qr rarer qr 2009

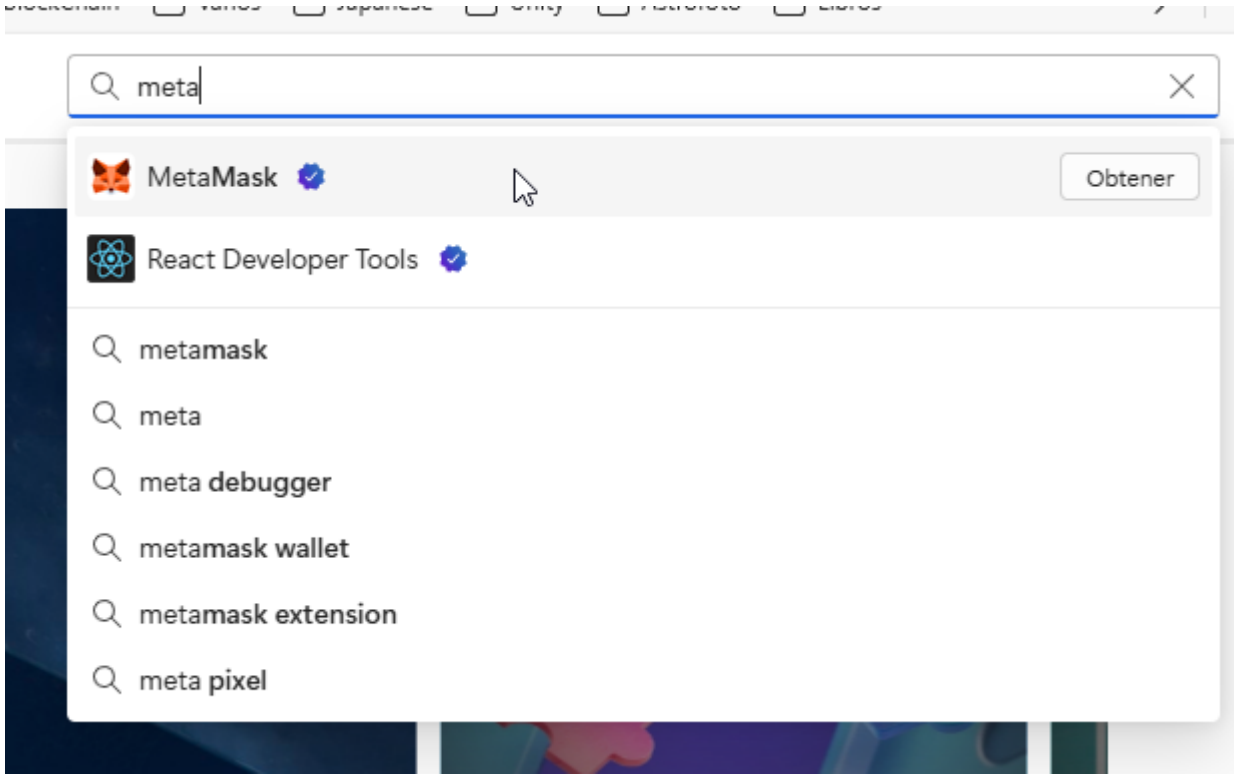
Misión:

- Descifrar el mensaje
- Una vez descifrado: ¿qué mensaje dejó Satoshi dentro de ese bloque y por qué es simbólico?

Metamask

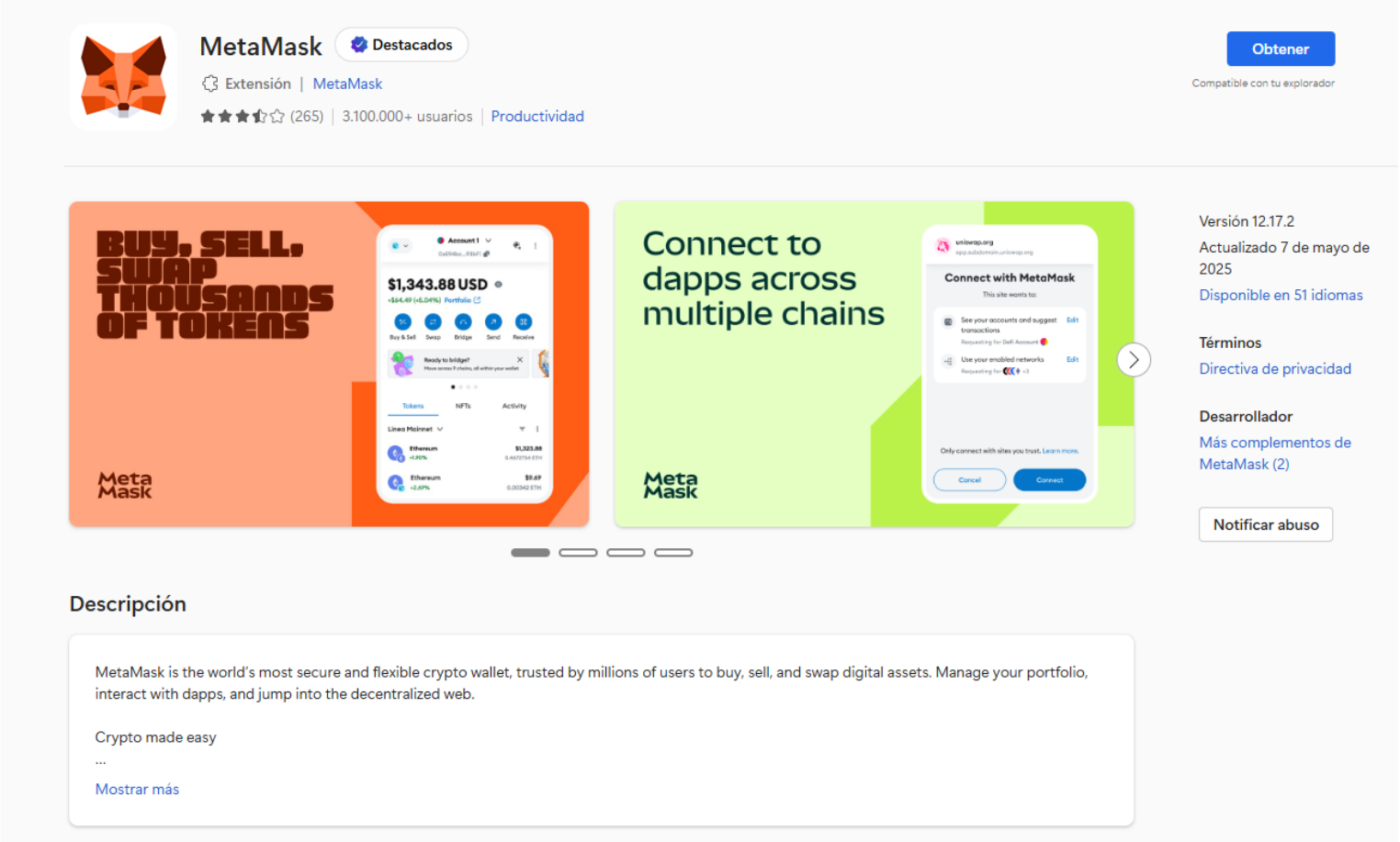
Metamask

- Ir a extensiones y buscar Metamask



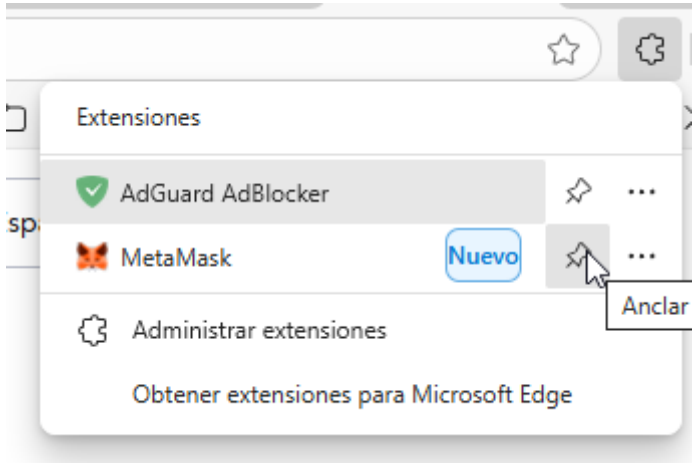
Metamask

- Click en obtener -> Agregar extensión



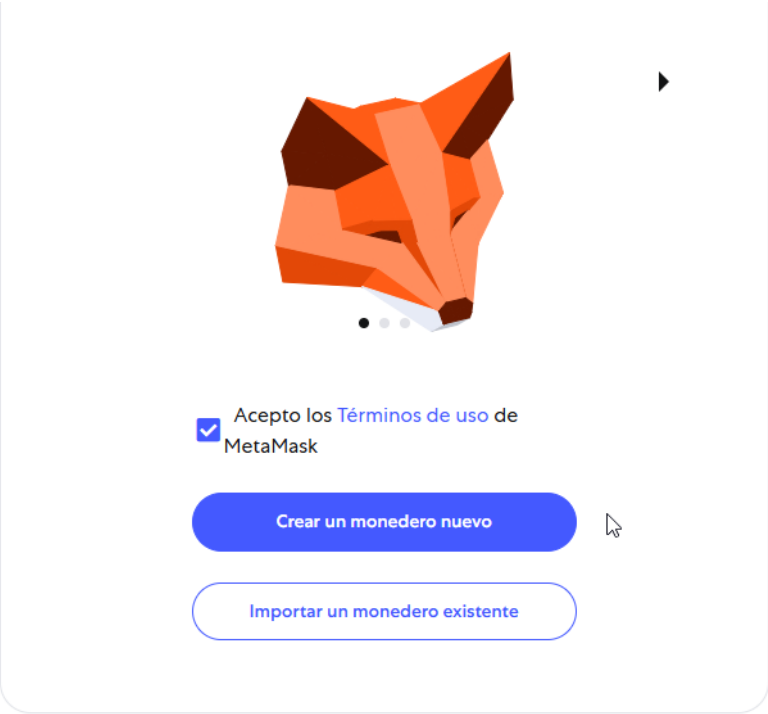
Metamask

- Desplegar las existentes y dejar fija Metamask por comodidad



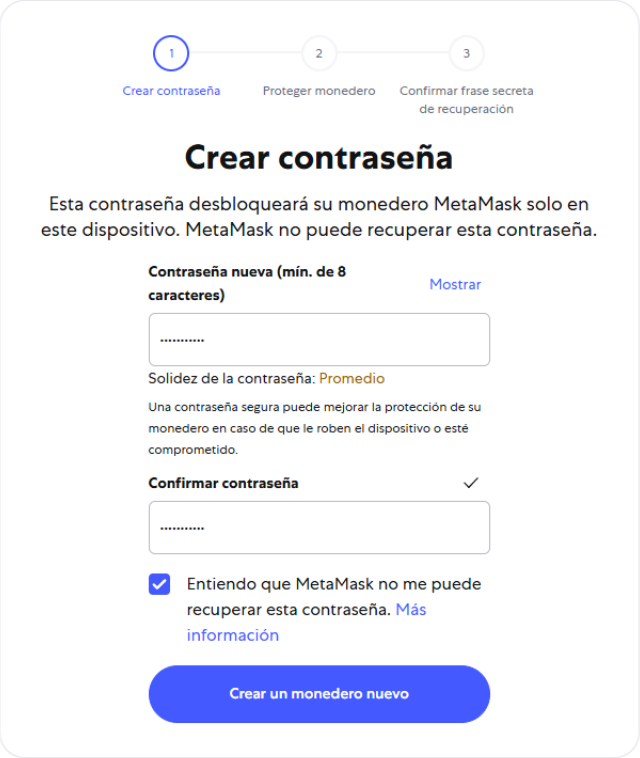
Metamask

- Aceptar términos y crear un monedero nuevo



Metamask

- Elegir una contraseña y **no olvidarla**

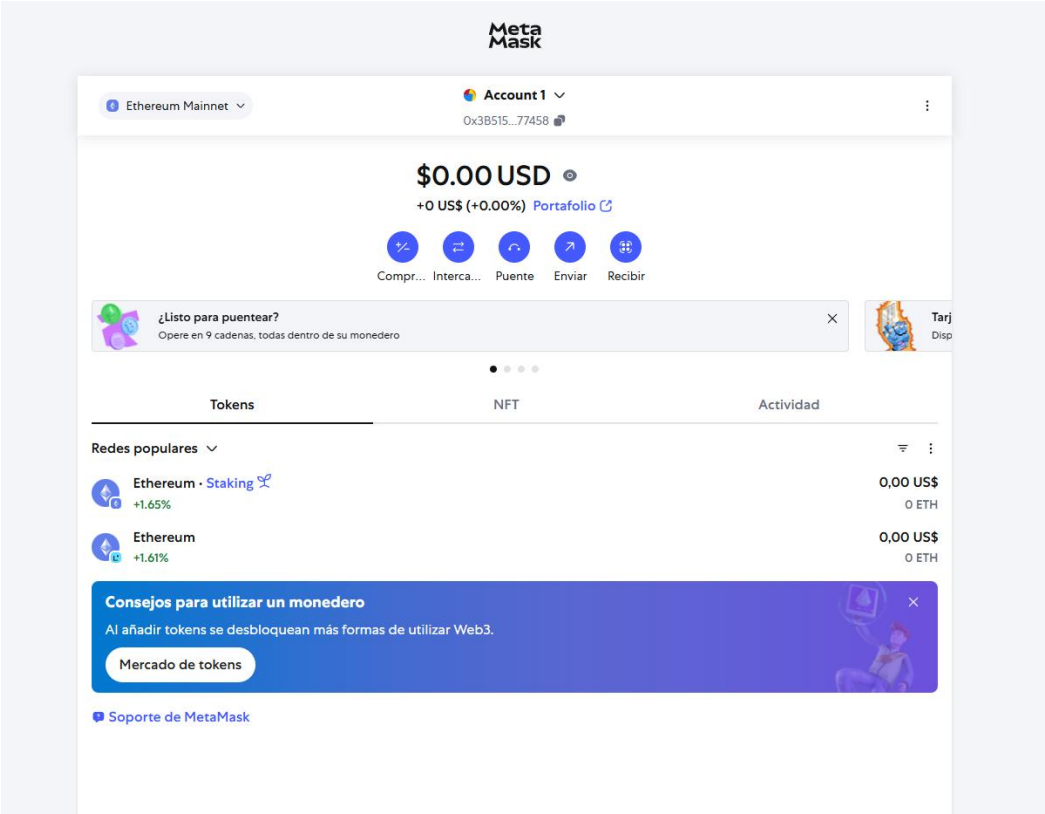


Metamask

- Elegir proteger el monedero, copiar las 12 palabras y NO PERDERLAS!!

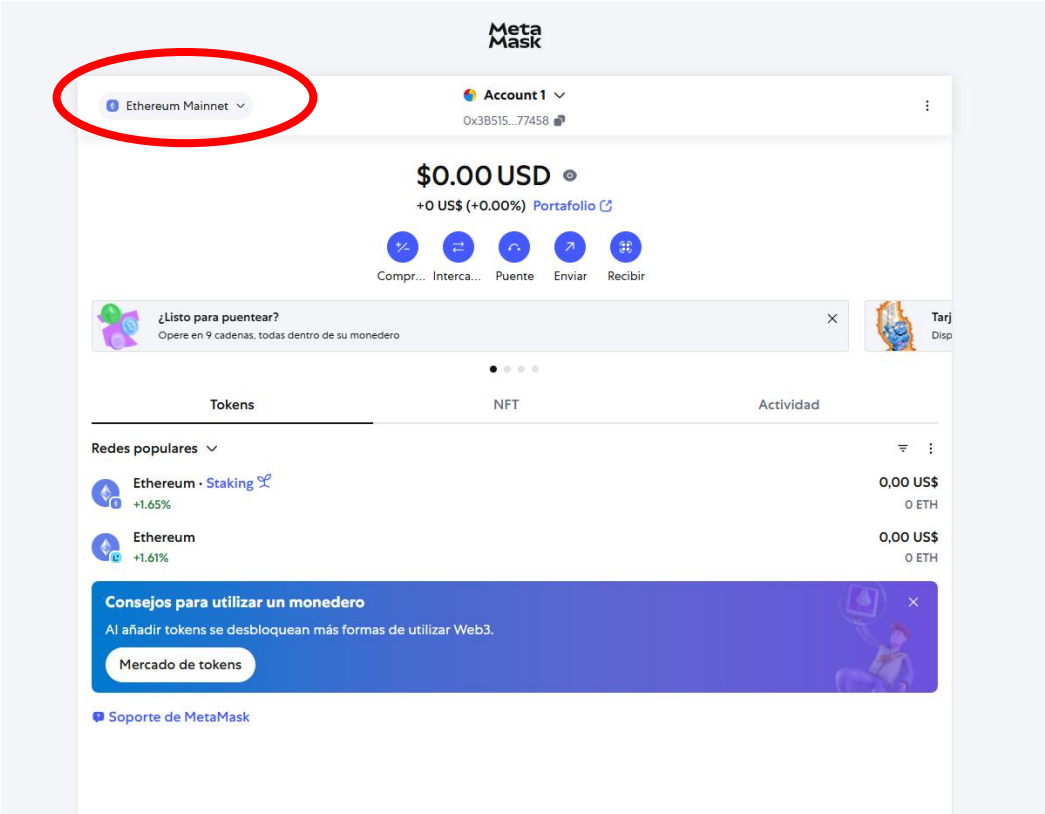
Metamask

- Esto es lo que deberéis ver



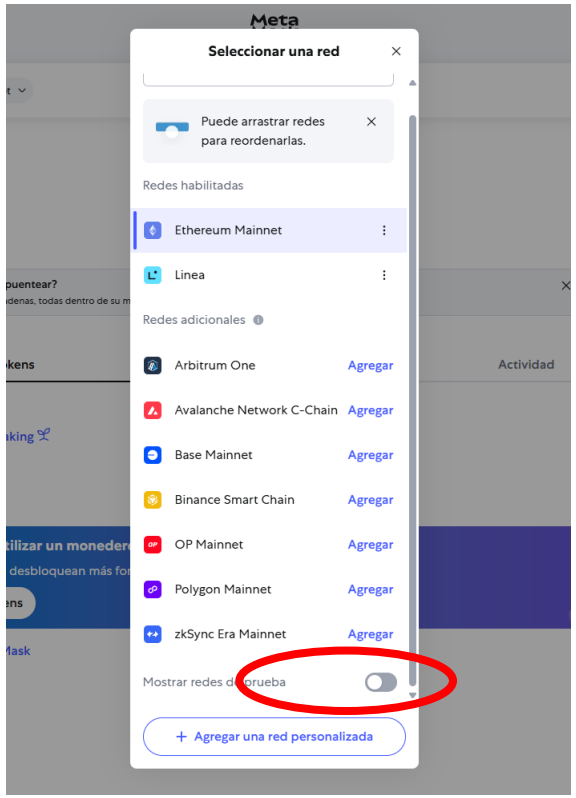
Metamask

- Despegar las redes



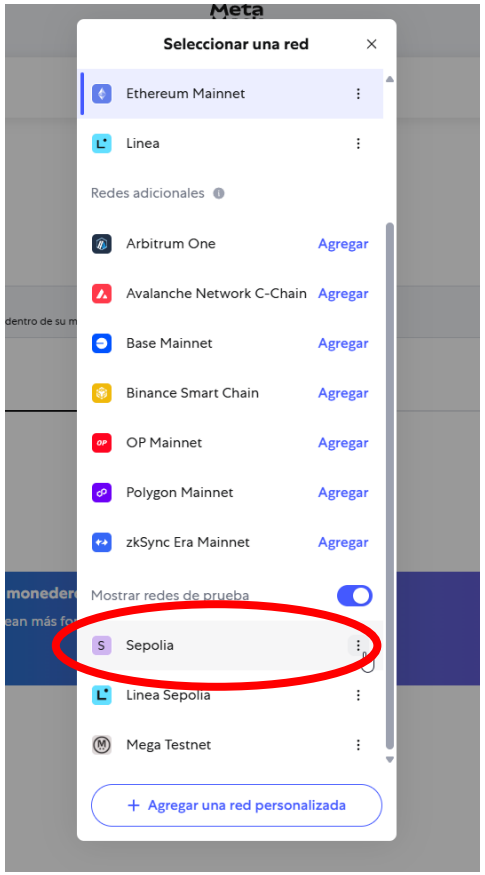
Metamask

- Macar ‘Mostrar redes de prueba’



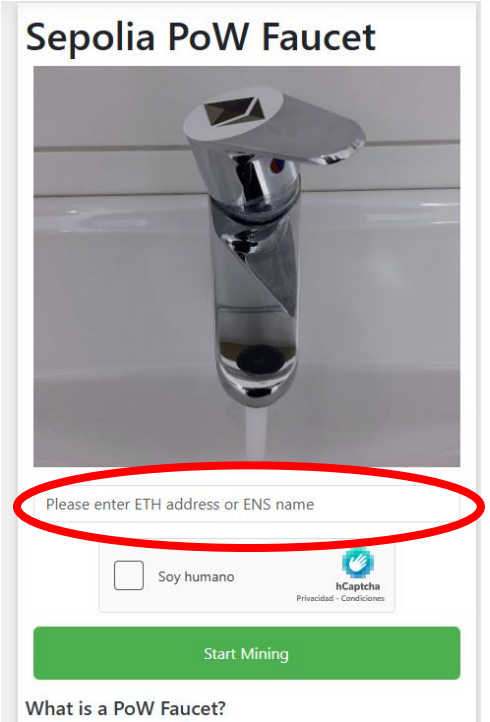
Metamask

- Click en Sepolia



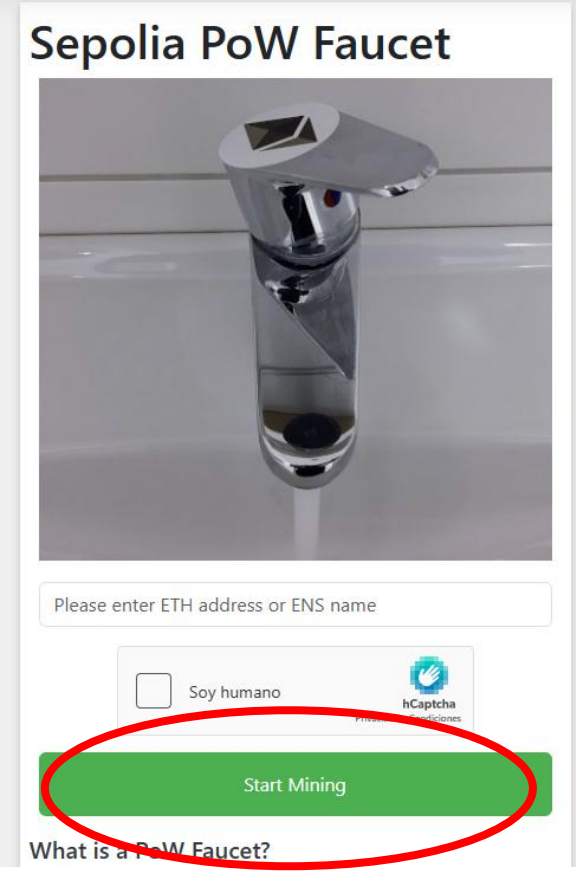
Metamask

- Vamos a conseguir unos cuantos eths de prueba
- Ir al faucet: https://sepolia-faucet.pk910.de/ y copiar nuestra address



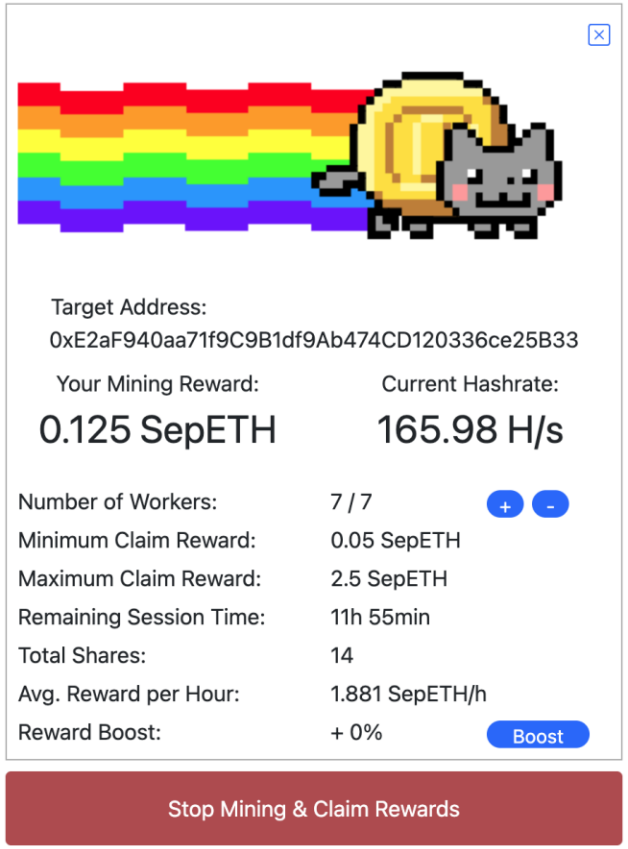
Metamask

- Rellenamos el captcha y pulsamos ‘Start Mining’



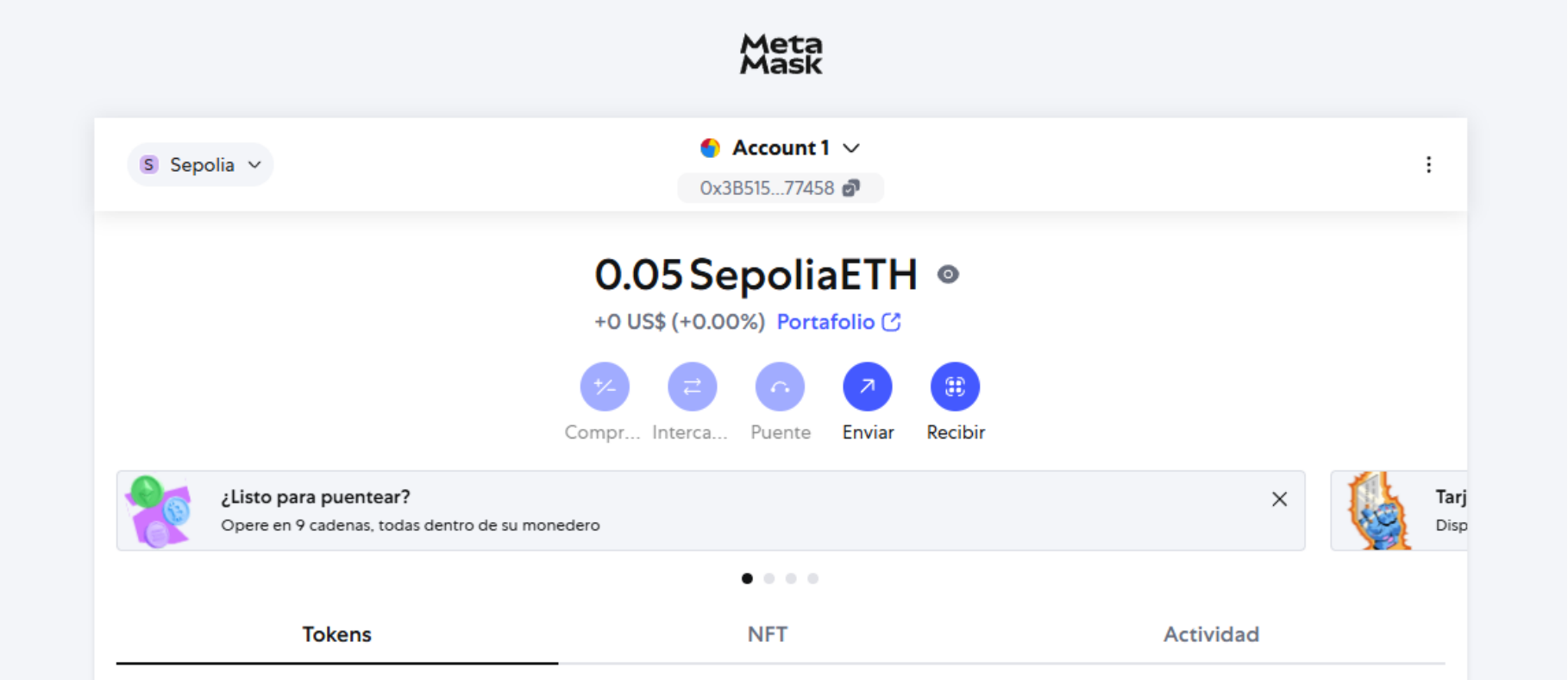
Metamask

- Cuando acumulemos suficiente cripto pulsamos ‘Stop Mining & Claim Rewards’



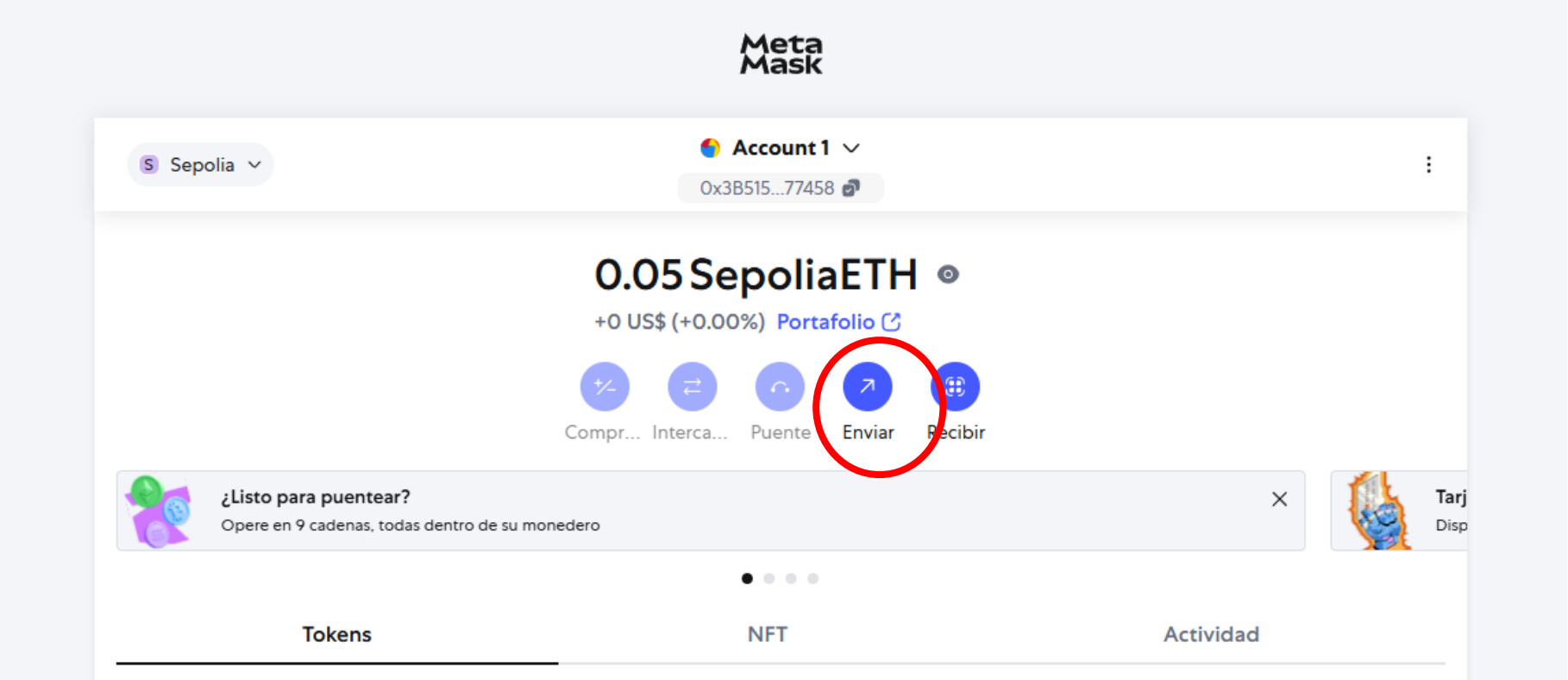
Metamask

- Y aparecerán en nuestra billetera Metamask



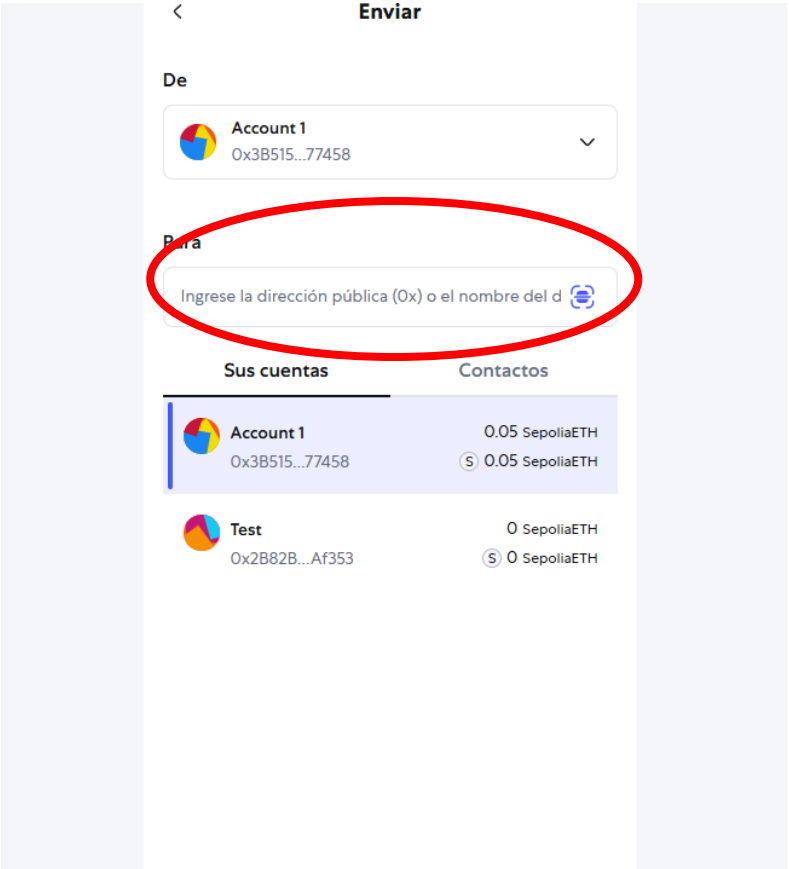
Metamask

- Pediremos a un compañero su dirección, la copiaremos al portapapeles y pulsaremos ‘Enviar’



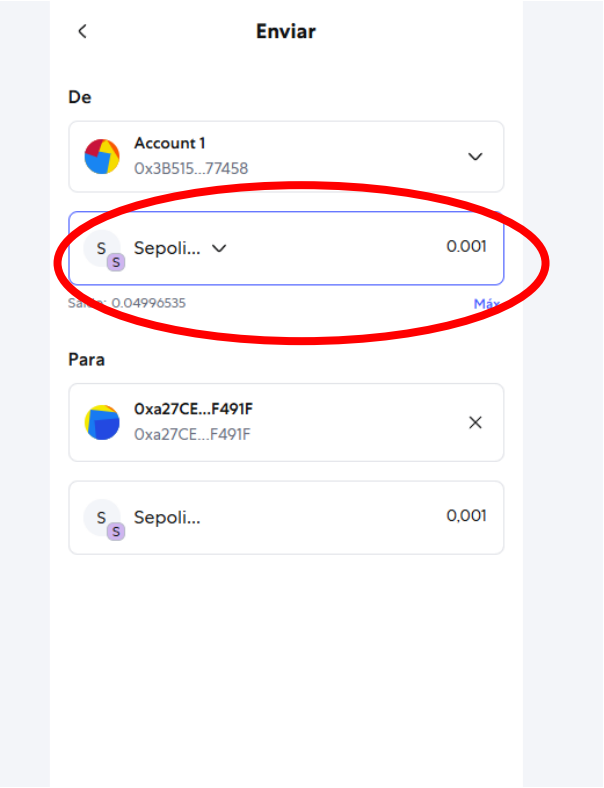
Metamask

- Pegamos su dirección en el campo ‘Para’



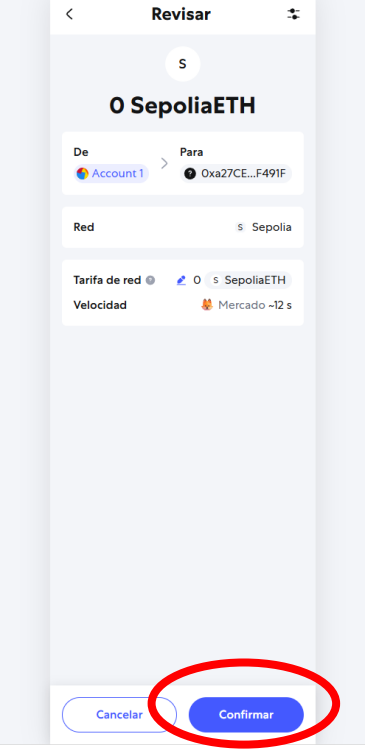
Metamask

- Indicamos la cantidad a enviar (por ejemplo 0.001) y damos continuar



Metamask

- Verificamos que todo es correcto y pulsamos ‘confirmar’

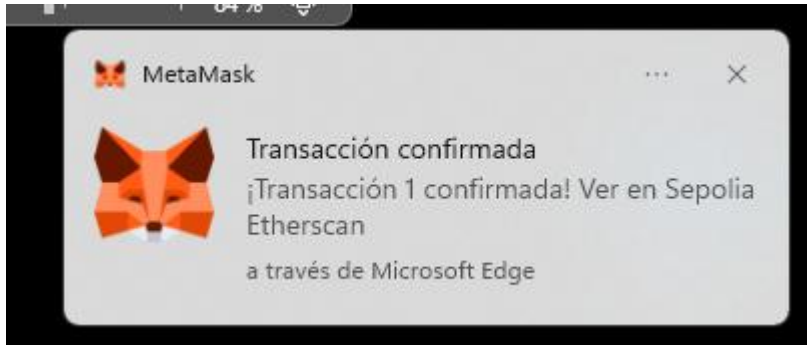


Metamask

- La transacción aparecerá como pendiente

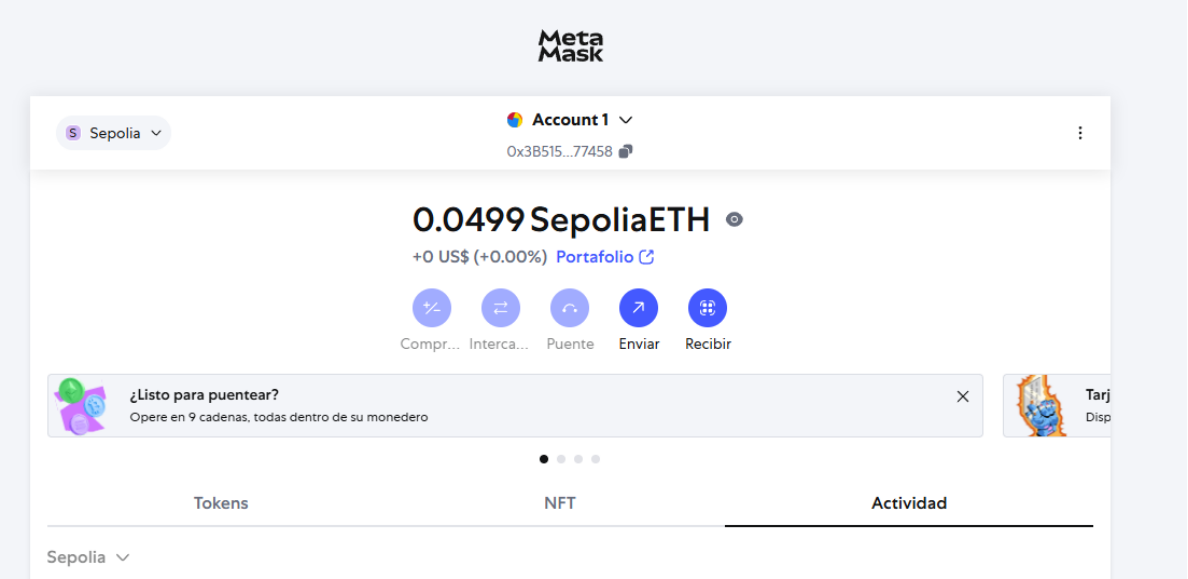
Metamask

- Y, finalmente aparecerá un pop-up avisando de que se ha finalizado



Metamask

- El cambio de saldo se verá reflejado en ambas billeteras



Metamask – Cambiar Fees

Hacer otra transacción

Al revisar los datos pulsar sobre el icono del lápiz que hay junto a ‘Tarifa de red’

Revisar las opciones existentes

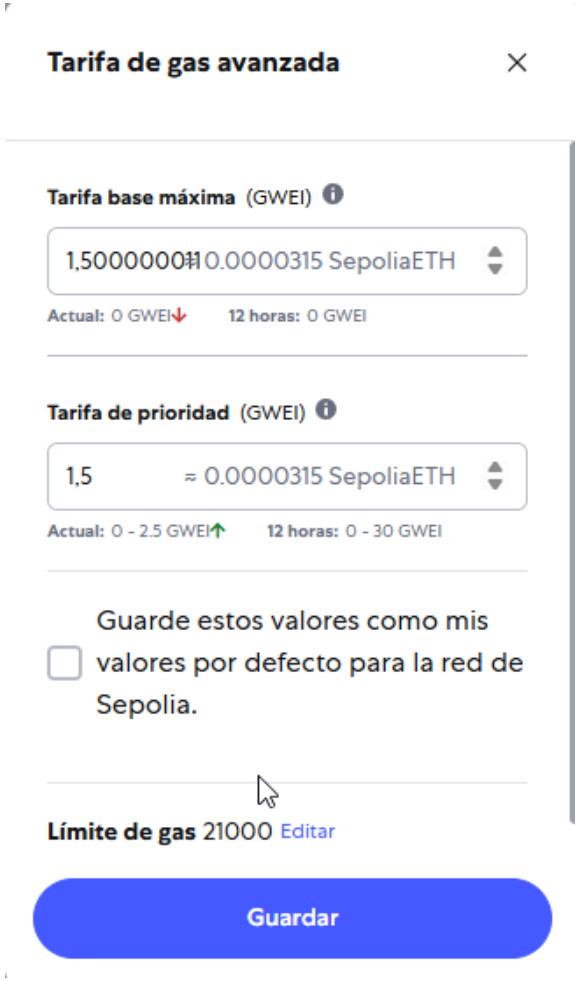
Ir a ‘avanzado’ y revisar las opciones que hay

Metamask – Introducción al Gas

Ir a ‘avanzado’ y revisar las opciones que hay.

- ¿Qué crees que es la ‘Tarifa base mínima’?

- ¿Y la ‘Tarifa de prioridad?



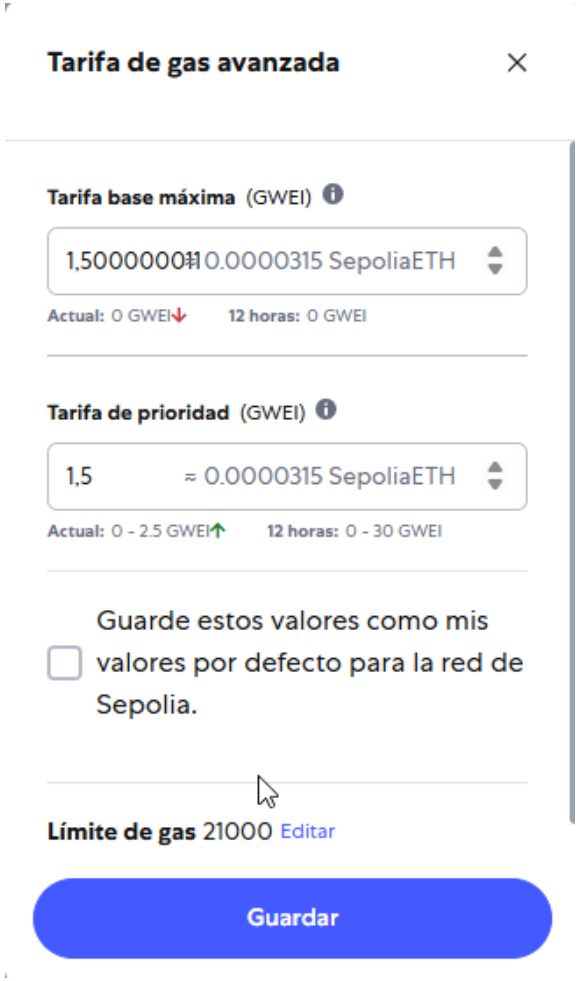
Metamask – Introducción al Gas

¿Qué crees que es la ‘Tarifa base mínima’?

- Es la cantidad máxima de gas (en GWEI) que estás dispuesto a pagar por unidad de gas para cubrir la tarifa base dinámica de la red.
- La red Ethereum (y testnets como Sepolia) ajusta una “base fee” automáticamente en cada bloque.
- Si la base real es menor que tu máximo, te devuelven la diferencia.

¿Y la ‘Tarifa de prioridad?

- Es la propina que ofreces al minero o validador para que priorice tu transacción.
- Cuanto mayor es, antes se confirma.



Metamask – Unidades en Ethereum

Unidad	Nombre	Equivalencia en Wei	Equivalencia en ETH	Uso típico
Wei	—	1	10^{-18} ETH	Unidad mínima, usada internamente
Kwei	Babbage	10^{3} Wei	10^{-15} ETH	Rara vez usada
Mwei	Lovelace	10^{6} Wei	10^{-12} ETH	Rara vez usada
Gwei	Shannon	10^{9} Wei	10^{-9} ETH	Unidad estándar para gas
Microether	Szabo	10^{12} Wei	10^{-6} ETH	Usada en algunos contratos antiguos
Milliether	Finney	10^{15} Wei	10^{-3} ETH	Usada en cálculos intermedios
Ether (ETH)	—	10^{18} Wei	1 ETH	Unidad principal para balances

Ejemplo:

- 1 Gwei = 0.000000001 ETH
- 21 000 gas × 30 Gwei = 630 000 000 000 000 Wei = **0.00063 ETH**

Discusión – Opciones Avanzadas

¿Qué pasa si mando una transacción con un fee muy bajo?

Discusión – Notas del profesor

¿Qué pasa si mando una transacción con un fee muy bajo?

La transacción entra al mempool pero los validadores la ignoran si otras pagan más, quedando “pendiente” indefinidamente.

Puede adolecer de inanición para siempre si las tarifas no bajan.

Mientras esté pendiente, bloquea las siguientes transacciones de esa cuenta ya que cada transacción lleva un número de secuencia.

Nonce de Ethereum

¿Qué es el Nonce en Ethereum?

Número secuencial que identifica cada transacción enviada por una cuenta.

Propósito

- Garantiza el **orden exacto** de las transacciones.
- Evita **repeticiones o duplicados** (ataques de replay).

Reglas clave

- Cada cuenta empieza en nonce = 0.
- Cada nueva transacción incrementa el nonce en +1.
- Las transacciones deben minarse **en orden**.
- Si se salta un número, las siguientes quedan **pendientes** hasta que la anterior se confirme.

Pregunta - Nonce de Ethereum

Si he mandado una transacción con un fee muy bajo y lleva atascada mucho tiempo (horas/días/meses) ¿Cómo la desbloquéo?

Pregunta – Notas del Profesor

Si he mandado una transacción con un fee muy bajo y lleva atascada mucho tiempo (horas/días/meses) ¿Cómo la desbloqueo?

Solución:

Identificar el nonce bloqueado

- Consultar el historial en Etherscan o en Metamask (Configuración avanzada → Nonce).

Reenviar (replace)

- Crear una nueva transacción con **el mismo nonce**.
- Enviar al **mismo destino** o incluso a tu **propia dirección**.
- Asignar un **gas más alto** para que los validadores la acepten.
- Esto sustituye la anterior (a esta técnica se la conoce como “Replace-by-fee”).

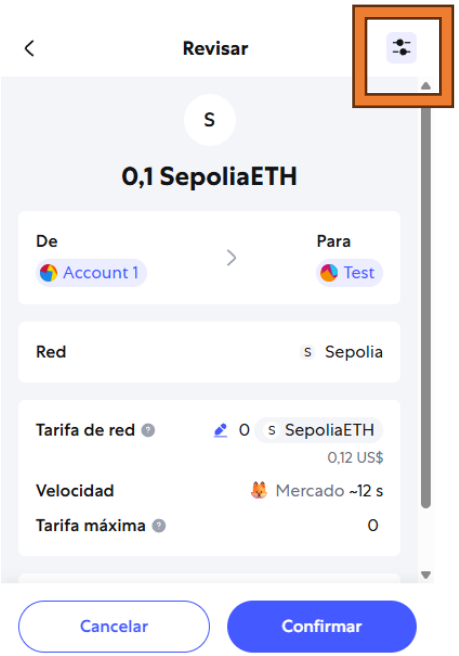
Cancelar (opcional)

- Enviar una transacción vacía (0 ETH a ti mismo) con el **mismo nonce** y gas alto.
- Si se mina, libera el nonce y desbloquea las siguientes.

Resultado:

La nueva transacción sobrescribe la antigua y la cola vuelve a fluir.

Metamask - ¿Dónde está el nonce?

Al llegar a la pantalla de ‘Confirmar’ pulsar sobre el icono de ‘settings’ (diales) que hay arriba a la derecha.

Aparecerá un campo editable con el nonce de la transacción a mandar.